



Config-Driven

Serwisy

Komponenty Aplikacji

- repozytorium: odczyt/zapis danych
- serwis: logika aplikacji / biznesowa
- validator: sprawdzanie poprawności obiektu
- UI: pobieranie i wyświetlanie danych
- main – uruchamia UI

<https://github.com/umcsuser/config-driven-demo.git>

Serwisy

Czego nie powinniśmy robić:

- ~~Dodawać logiki biznesowej w repo~~
- ~~Wstawiać logikę do UI.~~

Co powinniśmy zrobić?

- Użyć repo do zapisywania i odczytywania danych
- Użyć UI do pobrania danych od użytkownika i wyświetlania danych
- Użyć serwisów do logiki biznesowej

Serwisy

- Co to logika biznesowa? np. nie można usunąć pojazdu, jeśli jest wypożyczony

```
public void removeVehicle(String vehicleId) { 1 usage
    Vehicle vehicle = vehicleRepo.findById(vehicleId)
        .orElseThrow(() -> new IllegalArgumentException("Nie znaleziono pojazdu."));

    boolean rented = rentalRepo.findByVehicleIdAndReturnDateIsNull(vehicleId).isPresent();
    if (rented) {
        throw new IllegalStateException("Nie można usunąć pojazdu, bo jest aktualnie wypożyczony.");
    }
    vehicleRepo.deleteById(vehicle.getId());
}
```

Serwisy

- Serwisy będą używały repozytoriów do odczytywania/zapisywania danych
- Będą zwracały odpowiednie dane
- Funkcje biznesowe mogą być bardziej rozbudowane niż sam odczyt/zapis
- Repozytorium odpowiada tylko za dostęp do danych.

Serwisy

- Przykład wyświetlamy w UI w menu admina informację o pojazdach: dane pojazdu plus czy wypożyczony.
- Dwa sposoby na informację o wypożyczeniu:

```
case "1" -> vehicleService.findAllVehicles().forEach( Vehicle v ->
    System.out.println(v +
        " [Wypożyczony: " + vehicleService.isVehicleRented(v.getId()) + "]"
        + "[Wypożyczony: " + rentalService.vehicleHasActiveRental(v.getId()) + "]);
```

Serwisy

Funkcje w serwisach mogą wyglądać np:

- serwis VehicleService:

```
public List<Vehicle> findAllVehicles() { 1usage
    return vehicleRepo.findAll();
}
public boolean isVehicleRented(String vehicleId) { no usages
    return rentalRepo.findByVehicleIdAndReturnDateIsNull(vehicleId).isPresent();
}
```

- Serwis RentalService:

```
public boolean vehicleHasActiveRental(String vehicleId) { no usages
    return rentalRepo.findByVehicleIdAndReturnDateIsNull(vehicleId).isPresent();
}
```

Jakie kategorie pojazdów w systemie?

- Car, Motorcycle
- Mogą zostać dodane nowe np. Bus
- Zastąpiliśmy dziedziczenie i ify
- Ale brak walidacji
- Potrzebne szczegóły kategorii

Config-driven

Co to jest config-driven?

- struktura kategorii nie jest zaszyta na sztywno w klasach i ifach
- system czyta definicje kategorii z konfiguracji
- kod zna ogólne reguły, a szczegóły bierze z configu
- przenosi dane konfiguracyjne poza kod

Config-driven

Przykład:

- Vehicle jak na poprzednim wykładzie
- Dodajemy źródło kategorii – model + repo
- Dodajemy Walidator – czy pojazd jest poprawnie stworzony?
- Tworzymy serwis
- UI używa serwisu, który sprawdzi czy podany pojazd jest poprawny. Serwis używa repo aby zapisać pojazd.

Dostępne kategorie

```
1  [
2  {
3      "category": "Car",
4      "attributes": {
5          "fuelType": "string"
6      }
7  },
8  {
9      "category": "Motorcycle",
10     "attributes": {
11         "licence": "string"
12     }
13 },
14 {
15     "category": "Bus",
16     "attributes": {
17         "seats": "integer"
18     }
19 }
20 ]
```

- Kategorie umieszczamy w pliku json.
- Mamy informację o możliwych kategoriach pojazdów oraz ich dodatkowych atrybutach.
- Walidujemy po tym configu bez zmiany kodu.
- category — nazwa kategorii, np. Car
- attributes — mapa: nazwa atrybutu -> wymagany typ
- typy w przykładzie: string, number, boolean, integer

```

9      @Getter 29 usages
10     @Setter
11     @NoArgsConstructor
12     @ToString
13     public class VehicleCategoryConfig {
14         private String category;
15         @Getter(AccessLevel.NONE)
16         @Setter(AccessLevel.NONE)
17         private Map<String, String> attributes = new HashMap<>();
18         @Builder no usages
19         public VehicleCategoryConfig(String category, Map<String, String> attributes) {
20             this.category = category;
21             this.attributes = attributes == null ? new HashMap<>() : new HashMap<>(attributes);
22         }
23         public Map<String, String> getAttributes() { 2 usages
24             return Collections.unmodifiableMap(attributes);
25         }
26         > public void addAttribute(String name, String type) { attributes.put(name, type); }
29         > public void removeAttribute(String name) { attributes.remove(name); }
32         public VehicleCategoryConfig copy() { 2 usages
33             return VehicleCategoryConfig.builder()
34                 .category(category)
35                 .attributes(new HashMap<>(attributes))
36                 .build();
37         }
38     }

```

Repozytorium

```
8 public interface VehicleCategoryConfigRepository { 10 usages 2 implemer
9     List<VehicleCategoryConfig> findAll(); 1 usage 2 implementations
10    Optional<VehicleCategoryConfig> findByCategory(String category);
11 }
```

```
12 public class VehicleCategoryConfigJsonRepository implements VehicleCategoryConfigRepository {
13
14     private final JsonFileStorage<VehicleCategoryConfig> storage = 1 usage
15         new JsonFileStorage<> ( filename: "categories.json",
16                                 new TypeToken<List<VehicleCategoryConfig>>() {}.getType());
17
18     private final List<VehicleCategoryConfig> configs; 3 usages
19
20     public VehicleCategoryConfigJsonRepository() { 1 usage
21         this.configs = new ArrayList<>(storage.load());
22     }
```

Repozytorium

`@Override` 1 usage

```
24  
25 public List<VehicleCategoryConfig> findAll() {  
26     List<VehicleCategoryConfig> copy = new ArrayList<>();  
27     for (VehicleCategoryConfig config : configs) {  
28         copy.add(config.copy());  
29     }  
30     return copy;  
31 }
```

`@Override` 2 usages

```
32  
33  
34 public Optional<VehicleCategoryConfig> findByCategory(String category) {  
35     return configs.stream() Stream<VehicleCategoryConfig>  
36         .filter(VehicleCategoryConfig c -> c.getCategory() != null)  
37         .filter(VehicleCategoryConfig c -> c.getCategory().equalsIgnoreCase(category))  
38         .findFirst() Optional<VehicleCategoryConfig>  
39         .map(VehicleCategoryConfig::copy);  
40 }  
41 }
```

Serwis

```
8 public class VehicleCategoryConfigService { 11 usages
9
10     private final VehicleCategoryConfigRepository configRepository; 4 usages
11
12     public VehicleCategoryConfigService(VehicleCategoryConfigRepository configRepository) { 2 usages
13         this.configRepository = configRepository;
14     }
15
16     > public List<VehicleCategoryConfig> findAllCategories() { return configRepository.findAll(); }
17
18
19
20     public VehicleCategoryConfig getByCategory(String category) { 2 usages
21         return configRepository.findByCategory(category)
22             .orElseThrow(() -> new IllegalArgumentException("Nieznana kategoria pojazdu: " + category));
23     }
24
25     public boolean categoryExists(String category) { no usages
26         return configRepository.findByCategory(category).isPresent();
27     }
28 }
29
```

Walidator

- Walidator używa VehicleCategoryConfigService
- Walidator sprawdza poprawność obiektu Vehicle
- Przykładowe reguły:
 - cena nie może być ujemna
 - kategoria musi istnieć
 - wymagane atrybuty muszą być obecne
 - typ atrybutu musi zgadzać się z configiem

Przed wrzuceniem do repozytorium w serwisie (VehicleService) sprawdzimy walidatorem poprawność danych.

Validator

- Validator używa VehicleCategoryConfigService

```
8 public class VehicleValidator { 8 usages
9
10     private final VehicleCategoryConfigService configService; 2 usages
11
12     public VehicleValidator(VehicleCategoryConfigService configService) { 2 usages
13         this.configService = configService;
14     }
15 }
```

Validator

- Walidujemy podstawowe pola, plus atrybuty z mapy

```
16 public void validate(Vehicle vehicle) { 11 usages
17     if (vehicle == null) throw new IllegalArgumentException("Pojazd nie może być nullem.");
18
19     validateBaseFields(vehicle);
20     validateAttributes(vehicle.getAttributes(), configService.getByCategory(vehicle.getCategory()));
21 }
22
```

Validator

- Podstawowe pola:

```
private void validateBaseFields(Vehicle vehicle) { 1 usage
    requireNotBlank(vehicle.getCategory(), message: "Kategoria jest wymagana.");
    requireNotBlank(vehicle.getBrand(), message: "Marka jest wymagana.");
    requireNotBlank(vehicle.getModel(), message: "Model jest wymagany.");
    requireNotBlank(vehicle.getPlate(), message: "Numer rejestracyjny jest wymagany.");

    if (vehicle.getYear() <= 0) throw new IllegalArgumentException("Rok musi być dodatni.");
    if (vehicle.getPrice() < 0) throw new IllegalArgumentException("Cena nie może być ujemna.");
}
```

```
70  ~ private void requireNotBlank(String value, String message) {
71  ~     if (value == null || value.isBlank()) {
72         throw new IllegalArgumentException(message);
73     }
74 }
```

Validator

- Atrybuty z mapy:
- Mamy mapę z atrybutami pojazdu, oraz wzorcową mapę z atrybutami i jej typami dla danej kategorii

33 @

34

35

```
private void validateAttributes(Map<String, Object> actualAttributes, VehicleCategoryConfig config) {  
    Map<String, String> expectedAttributes = config.getAttributes();
```

Validator

- Atrybuty z mapy:
- Mamy mapę z atrybutami pojazdu, oraz wzorcową mapę z atrybutami i jej typami dla danej kategorii

```
33 @  
34 private void validateAttributes(Map<String, Object> actualAttributes, VehicleCategoryConfig config) {  
35     Map<String, String> expectedAttributes = config.getAttributes();
```

- Sprawdzenie czy nie ma nadmiarowych atrybutów:

```
37     for (String actualName : actualAttributes.keySet()) {  
38         if (!expectedAttributes.containsKey(actualName)) {  
39             throw new IllegalArgumentException("Nieobsługiwany atrybut dla kategorii "  
40                 + config.getCategory() + ": " + actualName);  
41         }  
42     }
```

Validator

- Sprawdzenie czy są wszystkie atrybuty i mają poprawny typ:

```
45  ~ expectedAttributes.forEach(( String attrName, String expectedType) -> {
46      Object value = actualAttributes.get(attrName);
47  ~  if (value == null) {
48      ~      throw new IllegalArgumentException("Brak wymaganego atrybutu: " + attrName);
49      ~  }
50  ~  if (expectedType.equalsIgnoreCase( anotherString: "string") && value instanceof String str) {
51      ~      requireNotBlank(str, message: "Atrybut " + attrName + " nie może być pusty.");
52      ~  }
53  ~  boolean isValidType = switch (expectedType.toLowerCase()) {
54      ~      case "string" -> value instanceof String;
55      ~      case "number" -> value instanceof Number;
56      ~      case "boolean" -> value instanceof Boolean;
57      ~      case "integer" -> value instanceof Integer;
58      ~      default -> throw new IllegalArgumentException("Nieobsługiwany typ w configu: " + expectedType);
59      ~  };
60  ~  if (!isValidType) {
61      ~      throw new IllegalArgumentException("Atrybut " + attrName + " musi być typu " + expectedType + ".");
62      ~  }
63  ~  });
64  ~ }
```

VehicleService

- Gdzie użyć Walidatora?
- Walidujemy gdy chcemy dodać auto.
- Walidacja jest logiką biznesową – dane dotyczące pojazdu muszą spełniać pewne warunki.
- Najlepiej użyć w serwisie

VehicleService

```
9 public class VehicleService { 6 usages
10
11     private final VehicleValidator vehicleValidator; 2 usages
12     private final VehicleRepository vehicleRepository; 3 usages
13
14
15     public VehicleService(VehicleValidator vehicleValidator, VehicleRepository vehicleRepository) {
16         this.vehicleValidator = vehicleValidator;
17         this.vehicleRepository = vehicleRepository;
18     }
19
20     public Vehicle addVehicle(Vehicle vehicle) { 1 usage
21         vehicleValidator.validate(vehicle);
22         vehicleRepository.save(vehicle);
23         return vehicle;
24     }
25     public List<Vehicle> findAllVehicles() { 1 usage
26         return vehicleRepository.findAll();
27     }
```


UI

- VehicleService używamy w UI.
- UI startujemy w funkcji main
- Zawartość jednej z funkcji: np. do dodawania pojazdu po pobraniu danych od usera:

```
67  
68      Vehicle added = vehicleService.addVehicle(vehicle);  
69      System.out.println("\nPojazd dodany pomyslnie:\n" + added);  
70      System.out.println(vehicleService.findAllVehicles());  
--
```

Main

- Przykład użycia

```
11 ▶ public class Main {  
12  
13 ▶     public static void main(String[] args) {  
14         VehicleCategoryConfigRepository configRepository = new VehicleCategoryConfigJsonRepository();  
15         VehicleCategoryConfigService configService = new VehicleCategoryConfigService(configRepository);  
16         VehicleRepository vehicleRepository = new VehicleRepoMocap();  
17         VehicleValidator validator = new VehicleValidator(configService);  
18         VehicleService vehicleService = new VehicleService(validator, vehicleRepository);  
19  
20         UI ui = new UI(configService, vehicleService);  
21         ui.start();  
22     }  
23 }
```

Dziękuję za uwagę